

도커의 동작 원리

1절 - 도커의 동작 원리

도커의 구조

● 도커의 구조



- 운영체제 위에 도커 엔진이 동작하고 그 위에서 컨테이너가 동작
- 프로그램과 데이터는 컨테이너 안에 위치

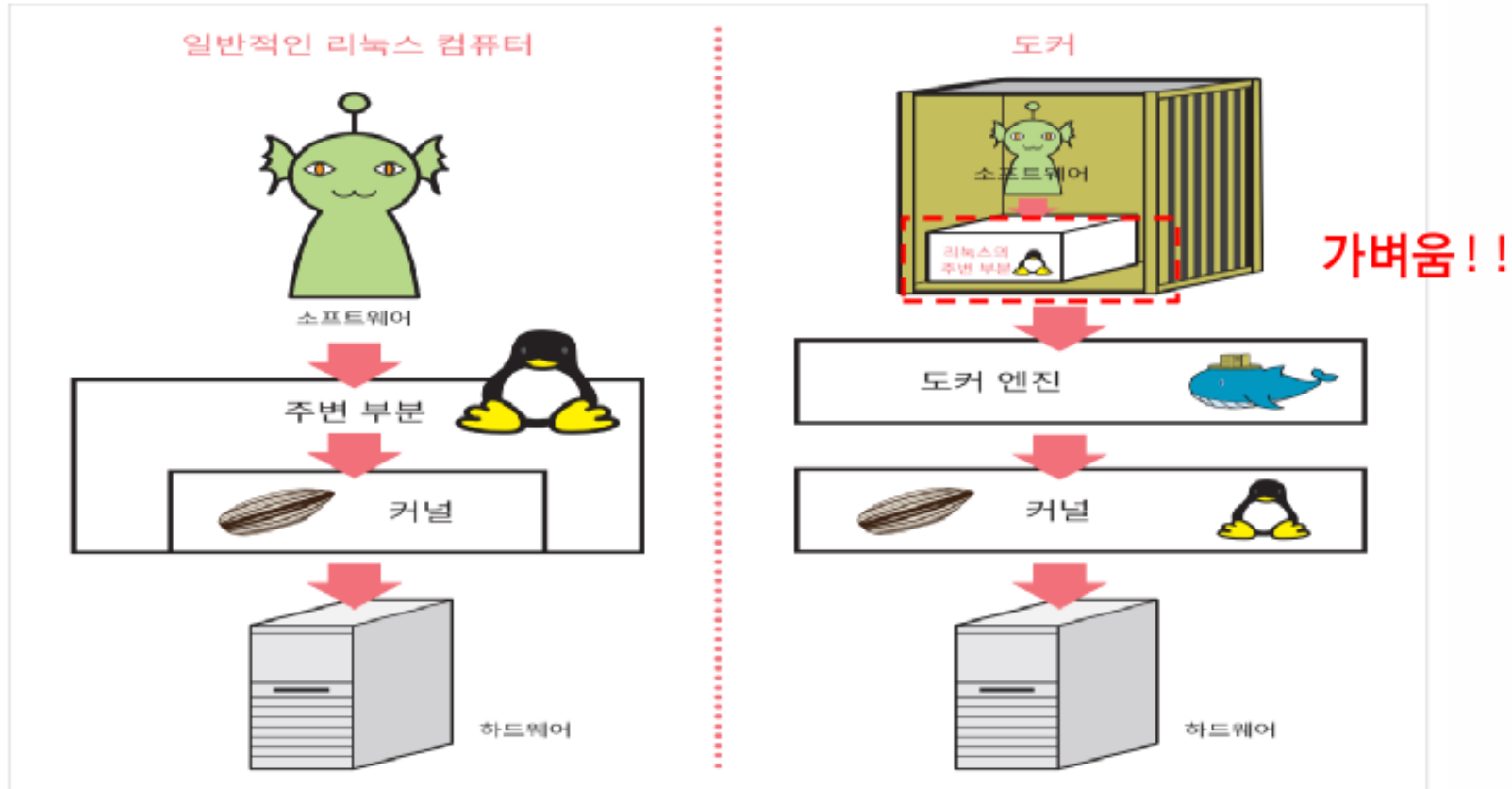
도커의 구조

- 모든 컨테이너 안에는 리눅스 운영체제와 유사한 환경 제공



도커의 구조

- 운영체제의 구조



- 컨테이너는 외부와 완전히 분리되어 있으므로 컨테이너 속에 운영체제의 주변 부분이 따로 들어 있다.

리눅스용 도커

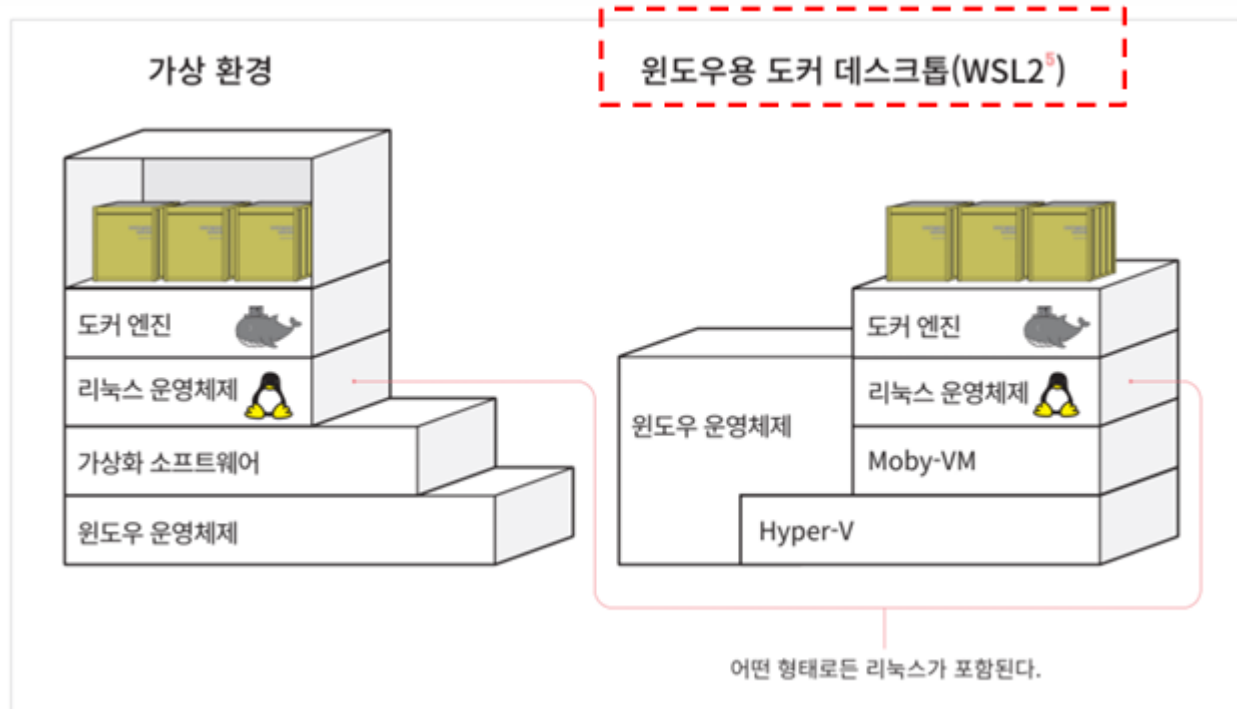
- 도커는 기본적으로 리눅스 운영체제에서만 동작
- 컨테이너에서 실행할 소프트웨어 역시 리눅스용 소프트웨어
- 즉, 도커는 리눅스 컴퓨터에 독립된 격리 환경을 만드는 것



- 컨테이너에 들어가는 프로그램은 리눅스용이어야 함

리눅스용 도커

- 윈도우와 macOS에서 도커 구동
- 윈도우나 macOS위에서 도커를 수행하려면 리눅스 운영체제를 갖추어야 함



- 도커는 기본적으로 리눅스용이다.

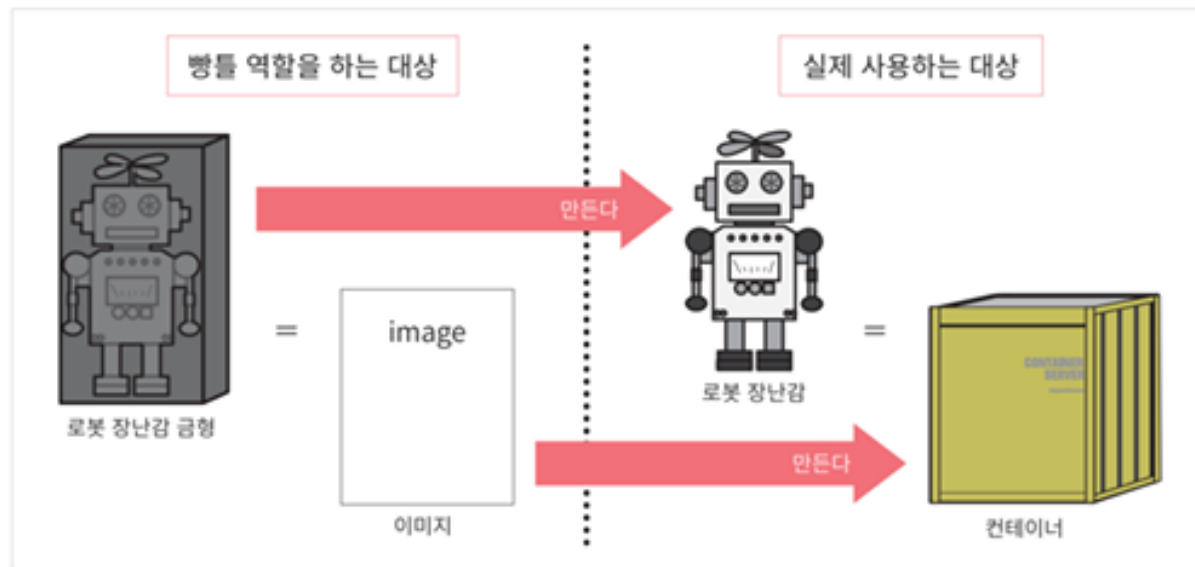
도커의 동작 원리

2절 - 도커 허브와 이미지, 그리고 컨테이너

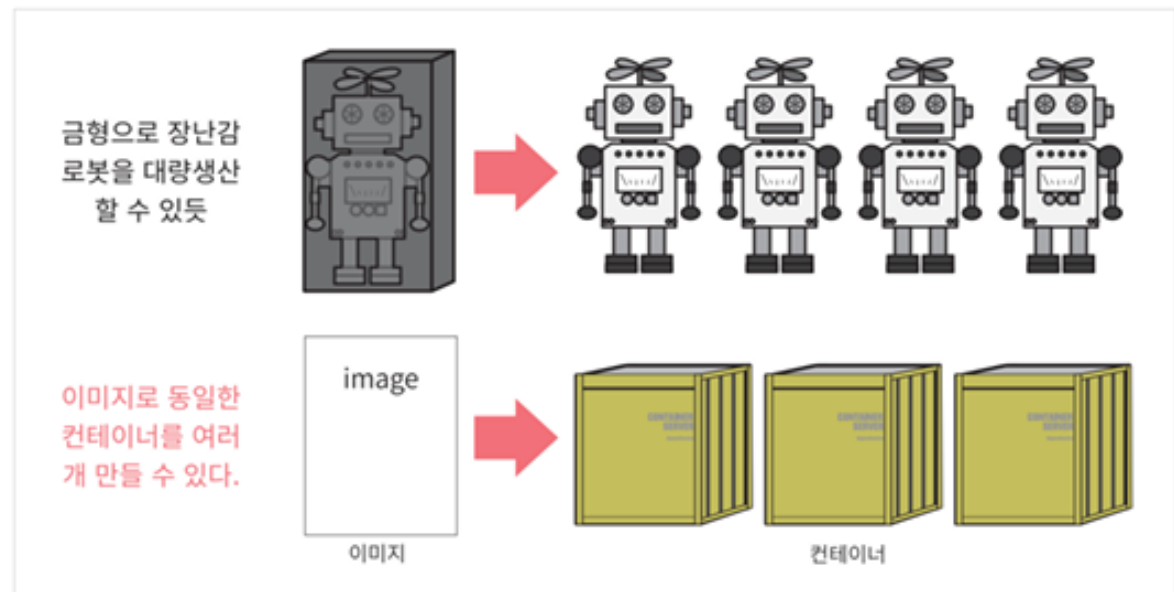
이미지와 컨테이너

- 이미지 : 컨테이너의 설계도

- 이미지는 컨테이너를 만드는 데 사용한다.

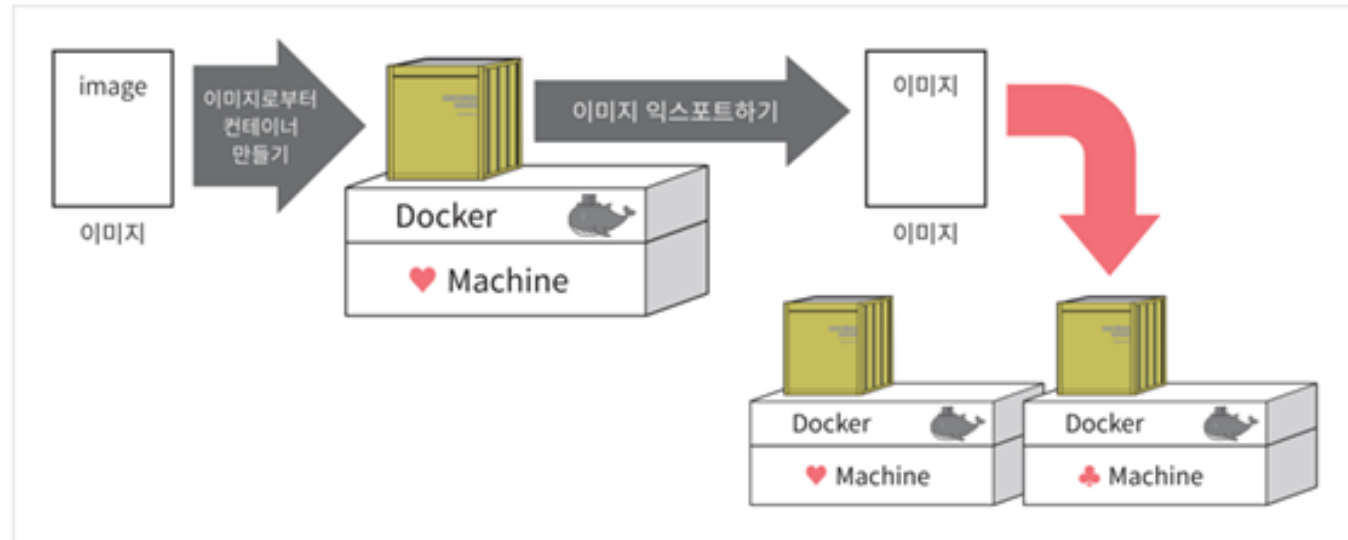


- 하나의 이미지로 동일한 컨테이너를 여러 개 만들 수 있다.



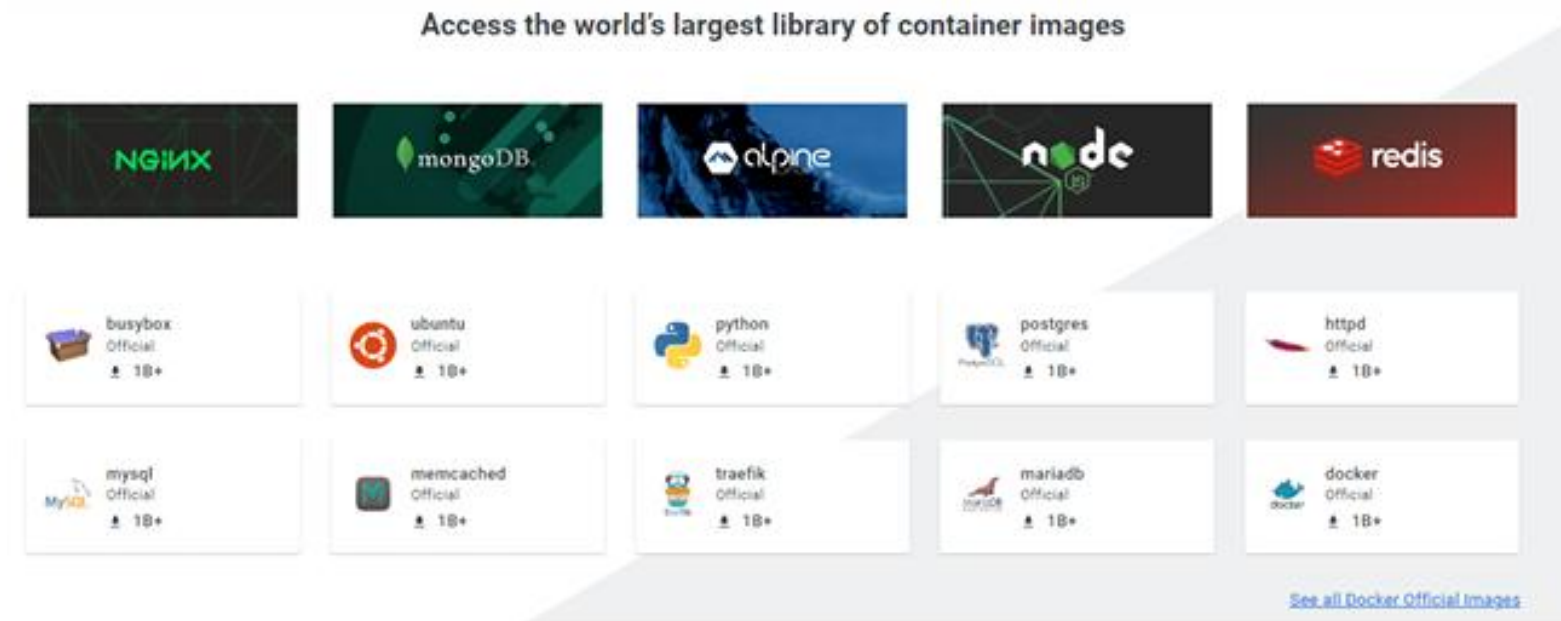
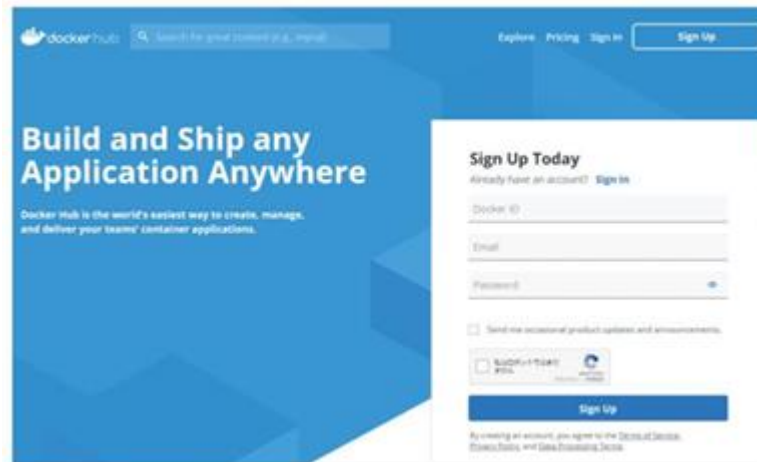
이미지와 컨테이너

- 컨테이너로 이미지를 만들 수 있다.
 - 새로운 이미지를 사용해 ‘개조된’ 컨테이너를 대량으로 만들 수 있다.
- 도커 엔진간 이동이 가능하다.
 - 컨테이너로부터 만든 이미지를 이용하면 다른 환경에서 원래의 컨테이너를 복원할 수 있다.



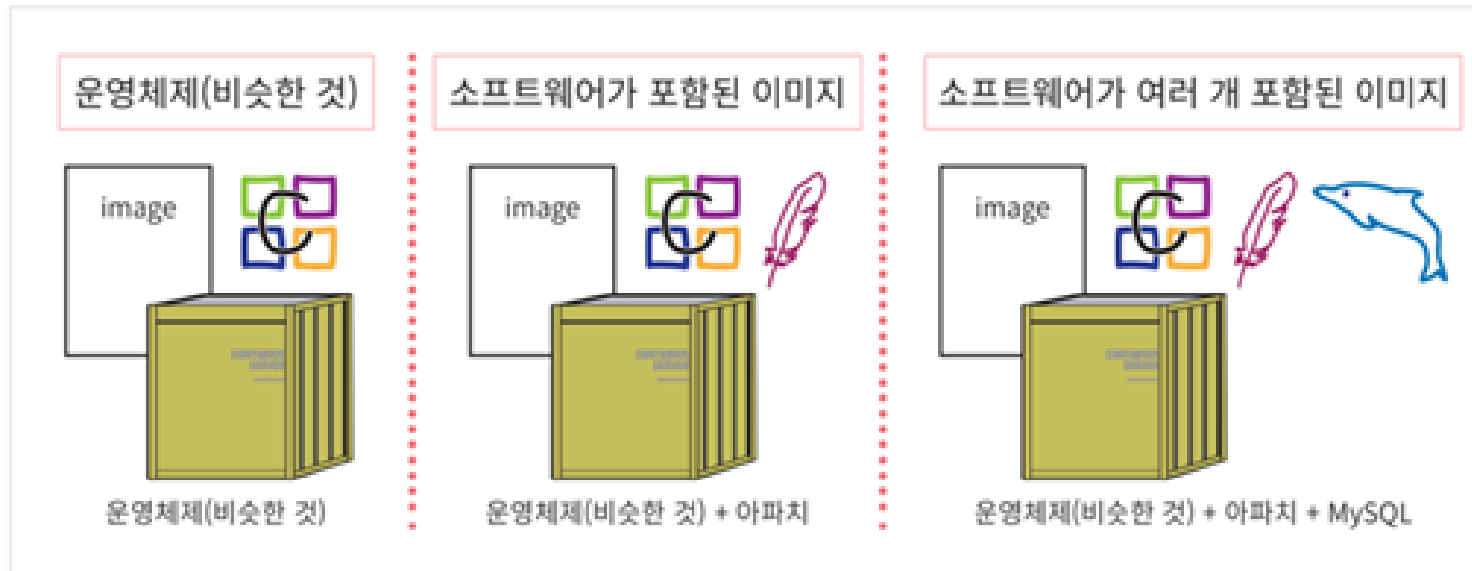
도커 허브와 도커 이미지

- 도커 허브(<https://hub.docker.com>)
 - 도커 레지스트리(이미지를 배포하는 사이트)
 - 공개된 컨테이너 이미지가 모여있는 사이트



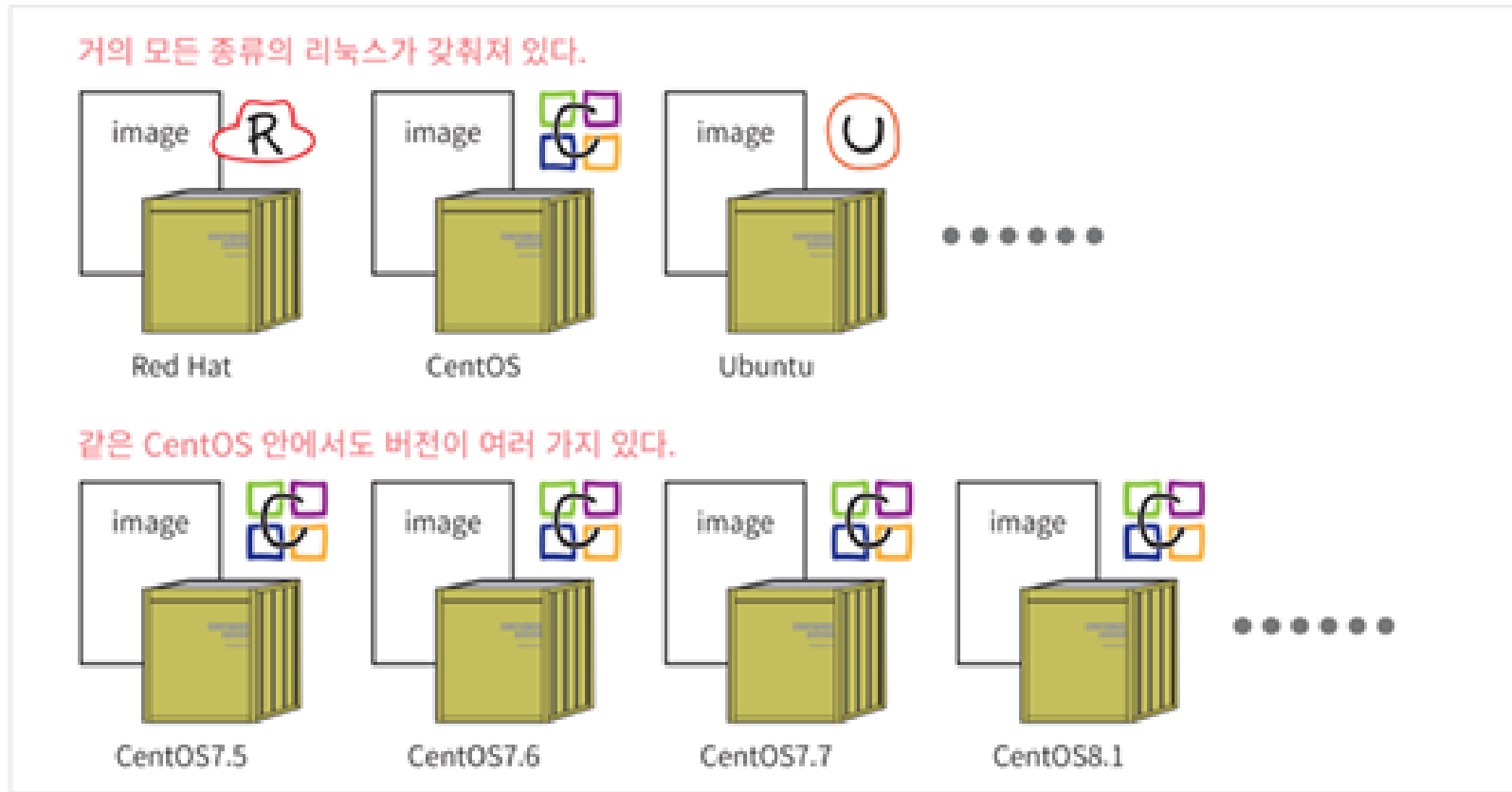
도커 허브와 도커 이미지

- 도커허브에서 제공하는 이미지
 - 도커 허브에는 다양한 이미지가 제공



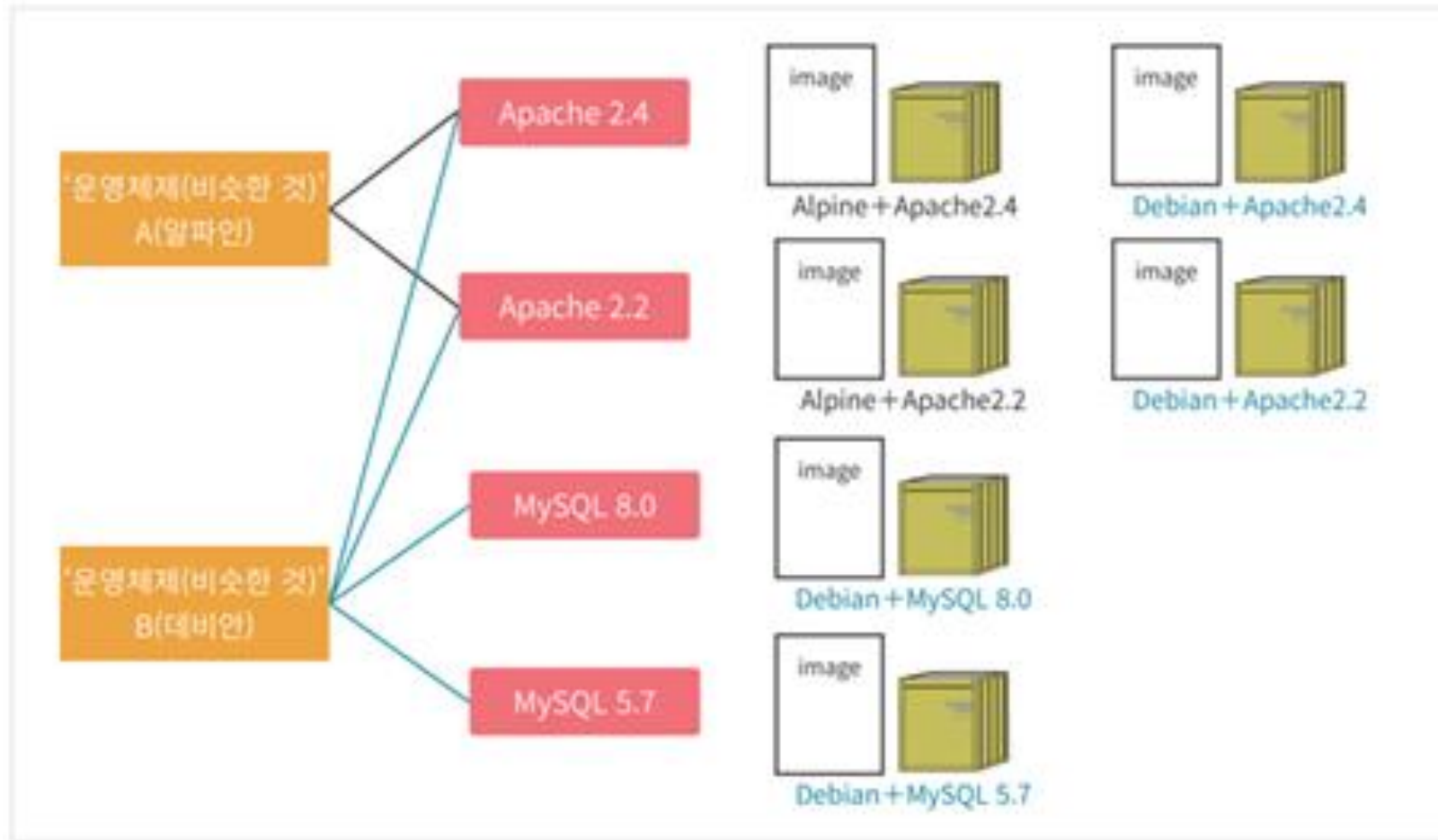
도커 허브와 도커 이미지

- 운영체제(비슷한 것)만 들어 있는 컨테이너도 여러 가지 존재



도커 허브와 도커 이미지

- 운영체제(비슷한 것)와 소프트웨어의 조합은 버전에 따라 더 다양해짐



도커 허브와 도커 이미지

- 안전한 컨테이너 이미지 고르는 방법
 - 공식 이미지를 사용한다
 - 커스텀 이미지를 직접 만들어 사용한다.
- 다양한 형태로 조합이 가능한 컨테이너
 - 도커 사용 원칙 ‘한 컨테이너에 한 프로그램’
 - 보안 및 유지 관리 측면에서 유리
 - 한 컨테이너에 필요한 프로그램을 모두 설치해도 가능

도커 허브와 도커 이미지

- 한 컨테이너에 하나의 프로그램 또는 여러 개의 프로그램을 넣을 수도 있다.

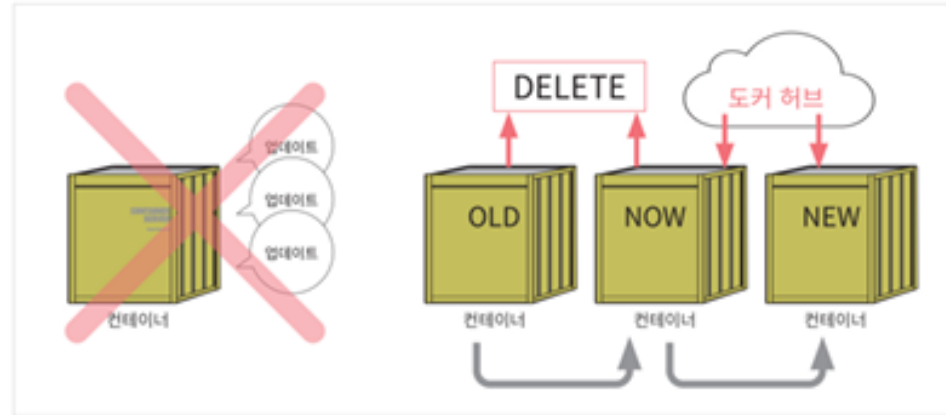


도커의 동작 원리

3절 - 도커 컨테이너의 생애주기와 데이터 저장

도커 컨테이너의 생애주기

- 도커 컨테이너는 일회용품으로 생각



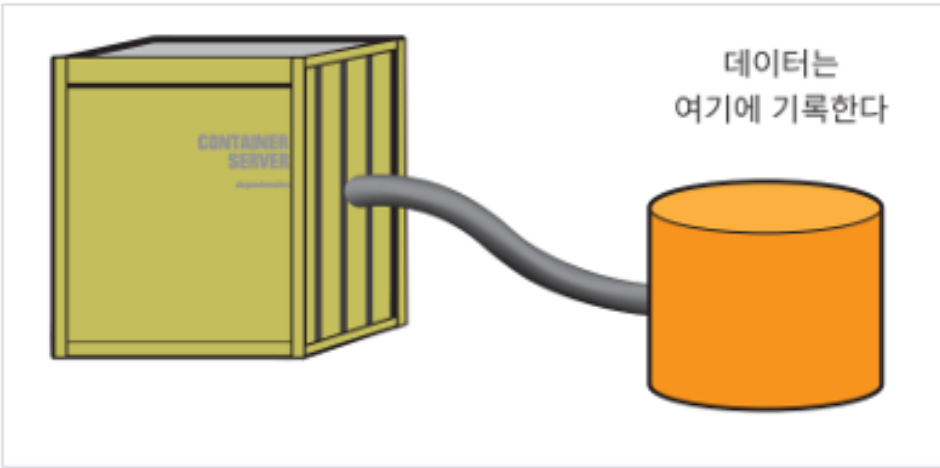
- 컨테이너는 일반적으로 여러 개를 동시에 가동하는 경우가 많으므로 유지보수가 용이하지 않을 수 있다.
- 따라서 업데이트시 새로운 이미지로부터 새로운 컨테이너를 만들어 배포하는 방식을 이용



컨테이너의 생애주기

데이터 저장

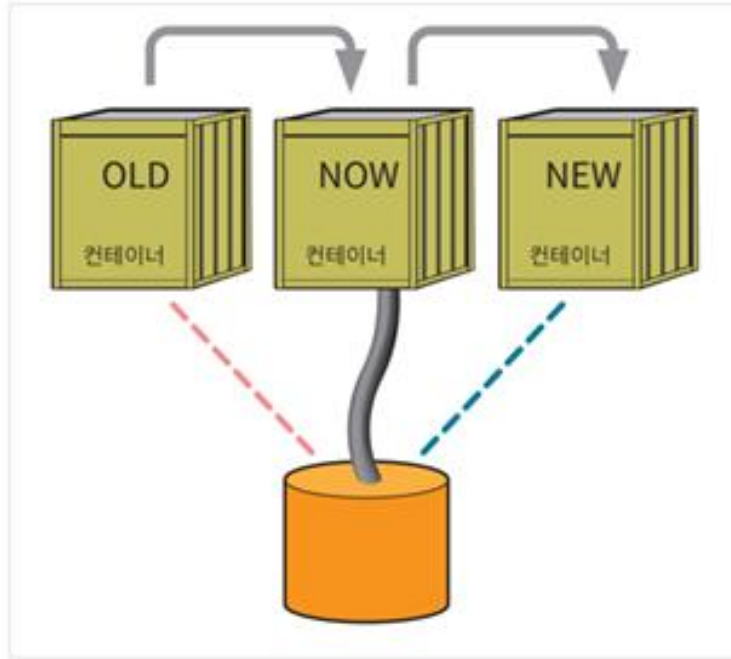
- 컨테이너 삭제시 그 안에 있던 파일도 함께 삭제
- 이를 방지하기위해 일반적으로 물리적 서버의 디스크를 마운트해 저장
- 마운트란 “디스크를 연결해 데이터를 기록할 수 있도록 한 상태”를 의미



- 물리적 컴퓨터의 일부를 마운트하고 여기에 데이터를 저장함
- 컨테이너를 삭제해도 데이터를 외부에 안전하게 보관
- 도커 엔진에 장애가 발생해도 데이터는 그대로 보존

데이터 저장

- 데이터의 외부 저장



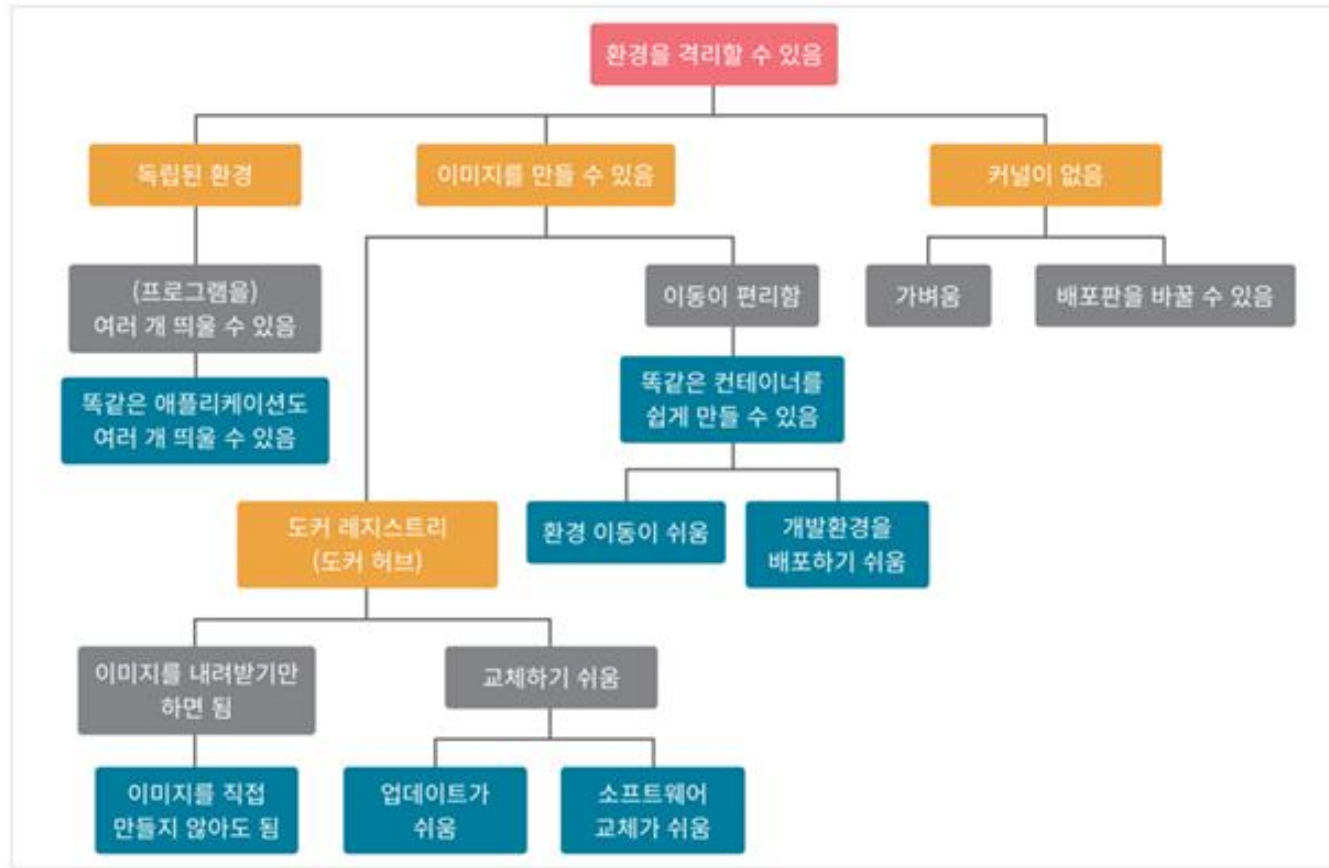
- 컨테이너가 바뀌어도 외부에 저장된 데이터는 계속 사용할 수 있다.
- 데이터나 설정 파일등은 외부에 저장
- 프로그램 개발 시 외부에 저장하지 않는 경우도 있으니 컨테이너 삭제 시 확인 필요

도커의 동작 원리

4절 - 도커의 장점과 단점

도커의 구조와 특성

● 도커의 성질



- 독립된 환경
- 이미지 생성 가능
- 컨테이너에 커널 포함되지 않아 가벼움

⇒ 격리 가능

도커의 장점과 단점

● 도커의 장점

- 1대의 물리적인 서버에 여러 대의 서버를 구동할 수 있다.
- 서버 관리가 용이하다
 - 다른 소프트웨어에 영향을 미치지 않음
 - 컨테이너 교체나 수정이 쉬워 환경 이전도 수월
- 서버 고수가 아니어도 다루기 수월
 - 명령 한줄로 서버 구축 완료

● 도커의 단점

- 리눅스 소프트웨어만 지원
- 호스트 서버에 문제발생시 모든 컨테이너에 영향을 미침
- 컨테이너를 하나만 사용하는 경우에는 비효율

도커의 주용도

- 팀원에게 동일한 개발환경 제공
 - 같은 프로젝트 팀원에게 개발환경을 동일하게 제공
 - 프로젝트별 컨테이너를 따로 사용할 수 있음
 - 개발 환경과 운영 환경의 차이가 없음
- 새로운 버전의 테스트
 - 새로운 운영체제나 라이브러리 버전을 개발환경에서 먼저 테스트하고 운영 환경에 컨테이너로 적용
- 동일한 서버가 여러 대 필요한 경우
 - 1대의 물리 서버에 여러 개의 컨테이너를 이용할 수 있으므로 비용 절약
 - 명령 한 줄로 원하는 만큼 서버 구동 가능
 - 스케일링에 유리하므로 웹 서버나 API 서버로 많이 활용